

Python 异步编程 — 问答问题集 (Q&A)

Python 异步编程 — 问答问题集 (Q&A)

配套文档: python-async.md 难度分层: 事实型 | 理解型 | 应用型**配套文档**: `python-async.md`
难度分层: 事实型 | 理解型 | 应用型
配套文档: python-async.md 难度分层: 事实型 | 理解型 | 应用型
配套文档: `python-async.md`
难度分层: 事实型 | 理解型 | 应用型

事实型问题 (Factual Questions)

- Q1: Python 中创建协程使用什么关键字? 交出控制权给事件循环使用什么关键字?
Q2: `asyncio.wait_for()` 和 `asyncio.wait()` 分别是什么? 哪个可以多次 `await`?
Q3: `asyncio.Semaphore(5)` 的作用是什么?
Q4: FastAPI 和 Uvicorn 相比 Flask 和 Gunicorn 的每秒请求数 (RPS) 大约是多少倍?
Q5: 在生产代码中, 应使用什么方法代替 `time.sleep()` 来避免阻塞事件循环?
Q6: 异步上下文管理器需要实现哪两个特殊方法?

理解型问题 (Comprehension Questions)

- Q7: 解释 GIL 对 Python 并发模型的影响。为什么多线程不适合 CPU 密集型但 `asyncio` 适合 I/O 密集型?
Q8: 代码中的生产者-消费者示例中, 为什么生产者最后放入了一个 `None` 值? 如果不放 `None` 会怎样?
Q9: 解释 `asyncio.gather()` 和 `asyncio.create_task()` 的区别和使用场景。
Q10: HTTP 客户端对比表中, 为什么 `aiohttp` 在 1000 个请求中仅需 2.1 秒, 而同步 `requests` 需要 65.3 秒?

应用型问题 (Application Questions)

- Q11: 你需要实现一个同时从 3 个不同 API 获取数据的接口, 任一 API 失败不应影响其他。请用 `asyncio` 设计实现。
Q12: 代码审查时发现同事在 `async def` 函数中直接调用了 `requests.get()`。这有什么问题? 应如何修改?
Q13: 设计一个基于拉取模型的系统, 支持添加任务、限制并发、超时取消、收集结果。
Q14: 为什么文档建议设置 `pythonasyncio.debug=1` 进行性能测试?
Q15: 为 FastAPI 应用添加请求超时控制。要求: 单请求最长处理时间 30 秒, 超过则返回 504。

自信度评级: 事实型 95%+ | 理解型 85%+ | 应用型 80%+ 问题覆盖: `async/await` 语法、并发模型对比、控制模式、性能数据、陷阱与最佳实践**自信度评级**: 事实型 95%+ | 理解型 85%+ | 应用型 80%+
问题覆盖: `async/await` 语法、并发模型对比、控制模式、性能数据、陷阱与最佳实践
自信度评级: 事实型 95%+ | 理解型 85%+ | 应用型 80%+ 问题覆盖: `async/await` 语法、并发模型对比、控制模式、性能数据、陷阱与最佳实践**自信度评级**: 事实型 95%+ | 理解型 85%+ | 应用型 80%+
问题覆盖: `async/await` 语法、并发模型对比、控制模式、性能数据、陷阱与最佳实践